sometimes called a *coroutine*. Fibers in Ruby are *cooperatively multitasked*, meaning that the responsibility for yielding control rests with the individual fibers and not the operating system. Fibers can explicitly yield control, or, in Ruby 3.0, be set to automatically yield control when its operations are blocked.

Fibers let you write programs that share control without incurring all of the complexity inherent in low-level threading. Let's look at a simple example. We'd like to analyze a text file, counting the occurrence of each word. We could do this (without using fibers) in a simple loop:

tut_threads/loop_word_count.rb

```
counts = Hash.new(0)
File.foreach("./testfile") do |line|
  line.scan(/\w+/) do |word|
    word = word.downcase
    counts[word] += 1
  end
end
counts.keys.sort.each { |k| print "#{k}:#{counts[k]} " }
```

produces:

```
and:1 is:3 line:3 on:1 one:1 so:1 this:3 three:1 two:1
```

However, this code is messy—it mixes word finding with word counting. We could fix this by writing a method that reads the file and yields each successive word. But fibers give us a simpler solution:

tut_threads/fiber_word_count.rb

```
words = Fiber.new do
  File.foreach("./testfile") do |line|
    line.scan(/\w+/) do |word|
      Fiber.yield word.downcase
    end
  end
  nil
end

counts = Hash.new(0)
while (word = words.resume)
  counts[word] += 1
end
counts.keys.sort.each { |k| print "#{k}:#{counts[k]} " }
```

produces:

```
and:1 is:3 line:3 on:1 one:1 so:1 this:3 three:1 two:1
```

The constructor for the Fiber class takes a block and returns a fiber object. Unlike a thread, the code in the block for a Fiber is not immediately executed.

After the Fiber is created, we can call resume on the fiber object. Calling resume the first time causes the block to start execution. In this case, the file is opened, and the scan method starts extracting individual words, and passing each individual word to the block passed to scan. Inside that block, Fiber.yield is called. Calling Fiber.yield suspends execution of the fiber—the resume method that we called to run the block returns any value passed to Fiber.yield.